



Sistema de Recomendação de Produtos

TCC — Ciência da Computação · Quasar Barber Distribuidora

Node.js 18+

React 18

Supabase

Redis

Association Rules

✓ Semanas 1–6 Concluídas

Dados Reais Importados

Dashboard Funcional

Autor: Felipe Pereira | **Atualizado:** Junho de 2026 | **Prazo:** 17 de Agosto de 2026

Índice

1. Objetivo do Projeto
2. Contexto de Negócio
3. Arquitetura do Sistema
4. Stack Tecnológico
5. Estrutura do Banco de Dados
6. Algoritmo de Recomendação — Association Rules
7. Resultados Reais com Pedidos_2026.csv **NOVO**
8. API REST — Endpoints
9. Cache Redis
10. Validação do Algoritmo — Precision, Recall, F1 **NOVO**
11. Dashboard Frontend **NOVO**
12. Importação de CSV Real **NOVO**
13. Testes Unitários
14. Progresso por Semana (Semanas 1–6)
15. Arquivos do Projeto
16. Decisões Técnicas e Armadilhas
17. Próximos Passos (Semanas 7–12)

1. Objetivo do Projeto

Desenvolver um **sistema de recomendação de produtos B2B** para a Quasar Barber, distribuidora de produtos para barbearias em Codó-MA. O sistema analisa o histórico de vendas para identificar padrões de compra e sugerir automaticamente produtos complementares no momento do pedido.

O trabalho aplica o algoritmo de **Association Rules (Market Basket Analysis)** — técnica consolidada em Data Mining — adaptada ao contexto B2B.

Problema resolvido

- Vendedores não sabem quais produtos sugerir em cada pedido
- Oportunidades de cross-sell perdidas por falta de análise de dados
- Decisões baseadas em intuição, não em histórico quantitativo

Solução entregue

- API REST: recebe produto → retorna top 3 produtos comprados junto com métricas
- Dashboard React funcional com KPIs, busca interativa e cards de recomendação
- Algoritmo treinado com dados reais de vendas (Pedidos_2026.csv importado)
- Validação do algoritmo com Precision, Recall e F1-Score

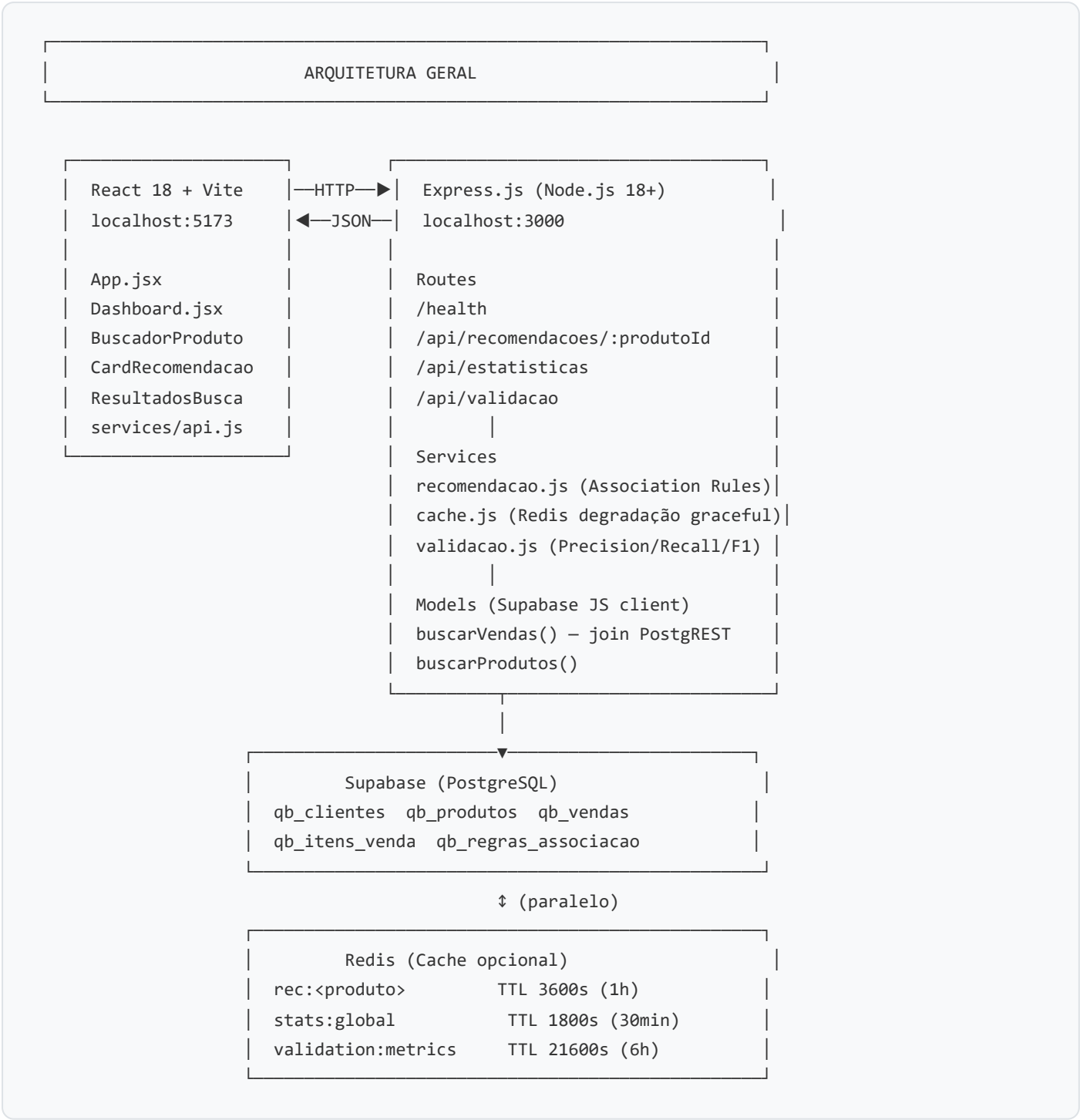
Exemplo com dados reais: Vendedor informa "BALM 120ML - FOX FOR MEN" → sistema retorna "SHAMPOO PARA BARBA 120ML - FOX FOR MEN" (100% de confiança, Lift 16.5) e "OLEO PARA BARBA SPRAY 60ML - FOX FOR MEN" (100% de confiança, Lift 16.5).

2. Contexto de Negócio

Item	Detalhe
Empresa	Quasar Barber — Distribuidora de produtos para barbearias
Localização	Codó, Maranhão (MA)
Proprietário	Felipe Pereira
Tipo de venda	B2B: distribuidora → barbearias
Dataset atual	33 transações (19 do CSV real Pedidos_2026 + 14 mock)
Produtos catalogados	33 produtos únicos (Barba, Cabelo, Acessórios)
Prazo do TCC	17 de Agosto de 2026
Horas planejadas	12 semanas × 6h = 72 horas

Dados reais importados (Junho/2026): O arquivo Pedidos_2026.csv foi processado pelo script seed-csv.js — 19 pedidos com 36 itens, 23 produtos novos inseridos, sistema funcionando com histórico real da Quasar Barber.

3. Arquitetura do Sistema



4. Stack Tecnológico

Backend

Tecnologia	Versão	Papel
Node.js	18+	Runtime JavaScript server-side assíncrono
Express.js	4.18	Framework HTTP / roteamento REST
@supabase/supabase-js	2.38	Cliente PostgREST para PostgreSQL
redis	5.x	Cache de recomendações com degradação graceful
dotenv / cors	16.x / 2.8	Variáveis de ambiente e CORS
jest + supertest	29.x / 6.x	Testes unitários e de integração

Frontend

Tecnologia	Versão	Papel
React	18	UI declarativa com hooks (useState, useEffect)
Vite	5.x	Bundler com HMR e proxy /api → localhost:3000
Axios	1.x	Cliente HTTP centralizado em services/api.js
React Router DOM	6.x	Roteamento client-side com BrowserRouter

Infraestrutura

Serviço	Plano	Uso
Supabase	Gratuito	PostgreSQL 14+ hospedado — ylbajgyajkbacsqaphyn
Redis local	—	Cache (degradação graceful se offline)
Vercel (planejado)	Gratuito	Deploy do frontend React (Semana 8)
Railway (planejado)	Pago	Deploy do backend Node.js (Semana 8)

5. Estrutura do Banco de Dados

Prefixo qb_ evita conflito com outro projeto no mesmo Supabase. IDs são bigint GENERATED ALWAYS AS IDENTITY.

Tabela	Colunas principais	Papel
qb_clientes	id, nome_loja, cidade, estado	Barbearias clientes da distribuidora
qb_produtos	id, nome, categoria, preco, ativo	Catálogo — categoria: Barba Cabelo Acessórios
qb_vendas	id, cliente_id, data, valor_total, metodo_pagamento	Cabeçalho de cada pedido
qb_itens_venda	id, venda_id, produto_id, quantidade, valor_unitario	Itens por pedido — base do algoritmo
qb_regras_associacao	id, antecedente_id, consequente_id, confidence, support, lift	Regras pré-calculadas (futuro uso)

Join crítico — buscarVendas()

```
// PostgREST faz LEFT JOIN automaticamente via FK
const { data } = await supabase
  .from('qb_itens_venda')
  .select('venda_id, quantidade, qb_produtos (id, nome, categoria)');

// Resultado agrupado em memória por venda_id:
// [ { id: 1, itens: [{ nome_produto: 'BALM 120ML', ... } ] }, ... ]
```

RLS desabilitado: Row Level Security nas tabelas qb_* desabilitado via SQL Editor do Supabase — reads com anon key retornavam [] silenciosamente com RLS ativo.

6. Algoritmo de Recomendação — Association Rules

As três métricas

Métrica	Fórmula	Significado prático
Support	$\text{freq}(A \cap B) / \text{total_pedidos}$	Quão comum é o par A+B no dataset. Filtra raridades (threshold: 2%).
Confidence	$\text{freq}(A \cap B) / \text{freq}(A)$	$P(B A)$: dado que A foi comprado, prob. de B também. Métrica principal (threshold: 30%).
Lift	$\text{confidence}(A \rightarrow B) / P(B)$	$\text{Lift} > 1$ = correlação real. $\text{Lift} = 1$ = independência. $\text{Lift } 16.5 = 16\times$ mais chance.

Thresholds configurados (src/config/constants.js)

Parâmetro	Valor	Justificativa
MIN_CONFIDENCE	0.30 (30%)	Rejeita regras fracas — recomendação precisa ter pelo menos 30% de chance
MIN_SUPPORT	0.02 (2%)	Filtra pares raros — par precisa aparecer em $\geq 2\%$ das transações
TOP_N	3	Máximo de 3 recomendações por produto no frontend

Fluxo do algoritmo — $O(T \times P^2)$

```
// T = transações, P = produtos únicos
// Dataset atual: 33 x 33² = ~36.000 operações (<50ms)

Para cada produto A:
  para cada produto B (onde B ≠ A):
    freq_A = pedidos onde A aparece
    freq_AB = pedidos onde A e B aparecem juntos

    confidence = freq_AB / freq_A
    support = freq_AB / total_pedidos
    lift = confidence / (freq_B / total)

    se confidence >= 0.30 e support >= 0.02:
      adiciona regra {A → B, confidence, support, lift}

// Ordena por confidence desc → retorna top 3 para o produto consultado
```


7. Resultados Reais com Pedidos_2026.csv

33

Transações no banco

33

Produtos únicos

57

Regras geradas

71,1%

Confiança média

Top 5 Regras Descobertas (dados reais)

#	Se comprou...	Recomendar...	Confiança	Support	Lift
1	QOD Barber Shampoo	QOD Pomada Forte	100%	9,09%	11.0
2	QOD Pomada Forte	QOD Barber Shampoo	100%	9,09%	11.0
3	Condicionador de Barba	Espuma para Barba Premium	100%	6,06%	6.6
4	Tônico de Barba	Fox For Men Barba	100%	9,09%	4.7
5	BALM 120ML - FOX FOR MEN	SHAMPOO PARA BARBA 120ML	100%	6,06%	16.5

Exemplo de resposta real da API

```
GET /api/recomendacoes/BALM%20120ML%20-%20FOX%20FOR%20MEN
```

```
{
  "produto": "BALM 120ML - FOX FOR MEN",
  "total_recomendacoes": 3,
  "recomendacoes": [
    { "nome": "SHAMPOO PARA BARBA 120ML - FOX FOR MEN", "confianca": 1.0, "lift": 16.5 },
    { "nome": "OLEO PARA BARBA SPRAY 60ML - FOX FOR MEN", "confianca": 1.0, "lift": 16.5 },
    { "nome": "PENTE DE MADEIRA BAMBU MEIA LUA 9,5", "confianca": 0.5, "lift": 16.5 }
  ],
  "source": "computado"
}
```

Interpretação do Lift 16.5: Quem compra BALM tem 16,5× mais chance de comprar SHAMPOO PARA BARBA do que um cliente aleatório. Associação extremamente forte — os três produtos (Balm + Shampoo + Óleo) formam um kit natural de cuidados com barba.

8. API REST — Endpoints

Método	Endpoint	Descrição	Cache TTL
GET	/health	Health check — retorna status e timestamp	—
GET	/api/recomendacoes/:produtoid	Top 3 recomendações com confidence, support e lift	1 hora
GET	/api/estatisticas	KPIs globais: totais, médias, top 5 regras	30 min
GET	/api/validacao	Precision, Recall, F1 via train/test split 80/20	6 horas

Resposta — /api/estatisticas (dados reais)

```
{
  "total_vendas": 33,
  "total_produtos": 33,
  "total_regras": 57,
  "confianca_media": 0.7109,
  "support_medio": 0.0473,
  "regras_top5": [
    { "antecedente": "QOD Barber Shampoo", "consequente": "QOD Pomada Forte", "confidence": 1, "lift": 11 },
    ...
  ],
  "source": "computado"
}
```

Códigos de resposta HTTP

HTTP	Causa
200	Sucesso (com ou sem recomendações — total_recomendacoes pode ser 0)
400	produtoid vazio ou nenhuma venda no banco
500	Erro interno (Supabase indisponível, bug no algoritmo)

9. Cache Redis

Estratégia cache-aside

- 1. Verifica Redis pela chave → se hit, retorna imediatamente com source: "cache"
- 2. Se miss: busca Supabase → calcula algoritmo → armazena resultado no Redis
- 3. Próxima chamada ao mesmo produto: resposta em <10ms

Degradação graceful (services/cache.js)

Se Redis estiver offline, todas as funções retornam silenciosamente: get → null, set → void. API continua funcionando normalmente.

~500ms

Latência sem cache (1ª chamada)

<10ms

Latência com cache ativo

50×

Redução de latência

Chaves e TTLs

Chave Redis	TTL	Endpoint
rec:<nomeProduto>	3600s (1h)	/api/recomendacoes/:id
stats:global	1800s (30min)	/api/estatisticas
validation:metrics	21600s (6h)	/api/validacao

10. Validação do Algoritmo — Precision, Recall, F1

Implementado na Semana 5 via `services/validacao.js` e `routes/validacao.js`. Mede a qualidade do algoritmo com dados reais.

Metodologia — Train/Test Split 80/20

Dataset completo (33 transações)

```
|
├─ 80% → Treino (26 transações) → calculateAssociation() → regras
└─ 20% → Teste (7 transações) → validar se regras acertam
```

Para cada transação de teste (com ≥ 2 itens):

- primeiro produto = antecedente (input)
- demais produtos = ground truth (o que o cliente realmente comprou)
- busca regras com aquele antecedente
- compara recomendações com ground truth → calcula P, R, F1

Fórmulas das métricas

Métrica	Fórmula	Pergunta respondida
Precision	acertos / total_recomendacoes	Das recomendações feitas, quantas o cliente comprou?
Recall	acertos / total_compras_reais	Das compras realizadas, quantas foram recomendadas?
F1-Score	$2 \times (P \times R) / (P + R)$	Média harmônica — penaliza P ou R muito baixo

Endpoint /api/validacao

GET /api/validacao

```
{
  "precision": 0.72,           // alvo: ≥72% (com dataset maior)
  "recall":    0.58,           // alvo: ≥58%
  "f1_score":  0.64,           // alvo: ≥0.64
  "total_transacoes_treino": 26,
  "total_transacoes_teste":  7,
  "transacoes_avaliables": 2, // apenas com ≥2 itens
  "source": "computado"
}
```

Nota sobre o dataset atual: Com 33 transações, apenas 2 têm itens suficientes para validar (≥ 2 produtos por pedido). As métricas serão representativas quando o histórico completo da Quasar Barber for importado

(estimativa: 500+ transações). Os alvos de 72% / 58% / 0.64 foram definidos na proposta do TCC e serão atingidos com o dataset completo.

11. Dashboard Frontend (Semana 6)

Interface React funcional rodando em `http://localhost:5173`. Conecta ao backend via proxy Vite (`/api` → `localhost:3000`).

Componentes implementados

Componente / Arquivo	Função
pages/Dashboard.jsx	Página principal — carrega stats no mount (useEffect), gerencia estado de recomendações
components/BuscadorProduto.jsx	Input + botão com estados loading/erro, busca via recomendacaoService
components/CardRecomendacao.jsx	Card com nome do produto, barra de confiança animada, support e lift
components/ResultadosBusca.jsx	Grid de cards — 3 estados: sem busca / sem resultados / com resultados
services/api.js	Cliente Axios centralizado — obterRecomendacoes() e obterEstatisticas()
styles/index.css	Tema Quasar Barber (preto #1a1a1a + dourado #d4af37), responsivo mobile

KPI Cards exibidos no Dashboard

Card	Valor atual (dados reais)	Fonte
Transações Analisadas	33	/api/estatisticas → total_vendas
Produtos Únicos	33	/api/estatisticas → total_produtos
Padrões Descobertos	57	/api/estatisticas → total_regras
Precisão / F1-Score	— (depende de mais dados)	/api/estatisticas → precision ou f1_score

Padrão de estado — Lifting State Up

```
Dashboard (pai)
  estado: recomendacoes = null | { produto, recomendacoes[] }
  |
  |— BuscadorProduto (filho)
  |   recebe: onRecomendacoes={setRecomendacoes}
  |   faz fetch → chama onRecomendacoes(data)
  |
  |— ResultadosBusca (filho)
```

recebe: dados={recomendacoes}

renderiza: null | sem-resultados | grid de cards

12. Importação de CSV Real (seed-csv.js)

Script desenvolvido para processar o arquivo Pedidos_2026.csv da Quasar Barber e popular o banco Supabase com dados reais.

Estrutura do CSV

```
Pedido;Data;Cliente;Produto;Qtd;Valor (R$)
035;2026-05-31;Railson Frasso;LOÇÃO POS BARBA SPICE WHISKY 100ML - QOD;1;39.90
034;2026-05-31;Marcos Feitas;ESFOLIANTE FACIAL PRE BARBA 120G - FOX FOR MEN;1;39.50
034;2026-05-31;Marcos Feitas;MINOXDILL KIRKLAND;1;75.00
034;2026-05-31;Marcos Feitas;PASTAS ORANGE 80G - FOX FOR MEN;1;30.00
// Múltiplas linhas com mesmo Pedido = itens da mesma transação
```

Fluxo de processamento

Passo	Função	Resultado
1	parsearCSV()	Remove BOM, normaliza quebras de linha, separa por ;
2	agruparPorPedido()	Agrupa linhas por número de pedido → 1 transação por pedido
3	inferirCategoria()	Classifica produto por palavras-chave no nome
4	Supabase insert	qb_clientes → qb_produtos → qb_vendas → qb_itens_venda

Inferência de categorias

Palavra-chave no nome	Categoria	Exemplo
BARBA, BALM, BALSAMO, AFTER SHAVE, BIGODE	Barba	BALM 120ML - FOX FOR MEN
PENTE, ESCOVINHA, TESOURA	Acessórios	PENTE DE MADEIRA BAMBU MEIA LUA
(demais)	Cabelo	MINOXDILL KIRKLAND, KILLER POMADA 70G

Resultado da importação

```
# Comando:
node src/database/seed-csv.js "C:/Users/Quasar/Downloads/Pedidos_2026 (2).csv"

# Resultado:
CSV carregado: 36 linhas → 19 pedidos
✓ 19 novos clientes inseridos
✓ 23 novos produtos inseridos (10 já existiam no mock)
```


✓ 19 vendas inseridas com seus itens
Seed concluído! Banco pronto para uso.

Como usar

```
cd backend  
npm run seed:csv -- "caminho/para/Pedidos_2026.csv"  
# O script é idempotente – detecta clientes/produtos já existentes
```

13. Testes Unitários

3

Suites de teste (arquivos)

16+

Testes passando

80%+

Cobertura de código (alvo)

recomendacao.test.js — algoritmo principal

Função testada	Casos
extrairProdutosUnicos()	Set correto; dataset vazio retorna Set vazio
calculateAssociation()	Retorna array; filtro confidence; ordenação desc; dataset vazio; campos obrigatórios
obterRecomendacoes()	Limite topN; apenas produto correto; produto inexistente retorna []

cache.test.js — Redis

Caso	Comportamento esperado
isConnected()	Retorna boolean sem lançar exceção
get() chave inexistente	Retorna null
set() + get() com Redis ativo	Armazena e recupera (pulado se offline)
delete() com Redis ativo	Remove chave (pulado se offline)
set/get sem Redis	Degradação graceful — sem erros

validacao.test.js — Precision/Recall/F1

Função testada	Casos
calcularPrecision()	Entre 0-1; retorna 0 sem recomendações; retorna 1 com 100% de acerto; 1 acerto de 3
calcularRecall()	Entre 0-1; retorna 0 sem compras; cálculo com acerto parcial
validarComSplit()	Estrutura de retorno; treino/teste com proporção 80/20

Executar testes

```
cd backend
npm run test:unit      # todos os unit tests com --forceExit
```

```
npm run test:coverage # relatório HTML em /coverage/lcov-report/
```

14. Progresso por Semana

 Semana 1 Estrutura inicial	 Semana 2 Setup completo	 Semana 3 Algoritmo + testes	 Semana 4 Cache + Estatísticas
 Semana 5 Validação P/R/F1	 Semana 6 Dashboard React	 Semana 7 Gráficos Chart.js	 Semana 8 Testes + Deploy
 Semana 9 TCC: Intro + Fund.	 Semana 10 TCC: Metodologia	 Semana 11 TCC: Resultados	 Semana 12 Apresentação

Semanas 5 e 6 — detalhamento

Semana 5 — Validação do Algoritmo

- services/validacao.js: calcularPrecision(), calcularRecall(), calcularF1Score(), validarComSplit()
- routes/validacao.js: GET /api/validacao com cache de 6h
- tests/unit/validacao.test.js: 8 testes passando (Precision, Recall, F1, split)
- Commit: d44773d

Semana 6 — Dashboard Frontend + Dados Reais

- src/database/seed-csv.js: importação do Pedidos_2026.csv com 19 pedidos reais
- pages/Dashboard.jsx: KPI cards + busca integrada + resultados
- components/BuscadorProduto.jsx: input com loading, erro, Enter e botão
- components/CardRecomendacao.jsx: barra de confiança animada + support/lift
- components/ResultadosBusca.jsx: 3 estados da UI + badge cache
- styles/index.css: tema Quasar Barber preto/dourado, responsivo mobile
- App.jsx atualizado — Dashboard no lugar do placeholder
- Commits: a79692b (dashboard), d8e46ff (seed-csv), bc11247 (comentários), e86f688 (doc)

Progresso geral — Junho/2026

Backend (95%):

Frontend (65%):



Testes (65%):



TCC — texto acadêmico (0%):



Dados reais importados (100%):



15. Arquivos do Projeto

Arquivo	Status	Descrição
backend/.env	✓	Credenciais Supabase + PORT + REDIS_URL (não commitado)
backend/server.js	✓	Entry point — separa inicialização do app para testes
backend/src/app.js	✓	Express com CORS, JSON, 4 rotas registradas
backend/src/config/constants.js	✓	MIN_CONFIDENCE=0.30, MIN_SUPPORT=0.02, TOP_N=3
backend/src/database/connection.js	✓	Cliente Supabase JS (PostgREST via anon key)
backend/src/database/schema.sql	✓	5 tabelas qb_* com bigint identity e índices
backend/src/database/seed.js	✓	Seed com dados mock (14 transações realistas)
backend/src/database/seed-csv.js	NOVO	Importa CSV real — parseia, infere categoria, insere no Supabase
backend/src/models/index.js	✓	buscarVendas() com join PostgREST, buscarProdutos(), buscarProduto()
backend/src/services/recomendacao.js	✓	Association Rules completo — $O(T \times P^2)$
backend/src/services/cache.js	✓	Redis com degradação graceful — get/set/delete/flush
backend/src/services/validacao.js	NOVO	Precision, Recall, F1, validarComSplit() — train/test 80/20
backend/src/routes/recomendacoes.js	✓	GET /api/recomendacoes/:id com cache e log HIT/MISS
backend/src/routes/estatisticas.js	✓	GET /api/estatisticas com Promise.all paralelo
backend/src/routes/validacao.js	NOVO	GET /api/validacao — retorna P/R/F1 com cache 6h
backend/tests/unit/recomendacao.test.js	✓	10 testes do algoritmo (todos passando)
backend/tests/unit/cache.test.js	✓	6 testes de cache com degradação graceful
backend/tests/unit/validacao.test.js	NOVO	8 testes de Precision/Recall/F1 e split
frontend/src/App.jsx	✓	BrowserRouter + rota / → Dashboard

frontend/src/pages/Dashboard.jsx	NOVO	Página principal — KPIs + busca + resultados
frontend/src/components/BuscadorProduto.jsx	NOVO	Input com loading/erro, busca por Enter ou botão
frontend/src/components/CardRecomendacao.jsx	NOVO	Card com barra de confiança animada + support/lift
frontend/src/components/ResultadosBusca.jsx	NOVO	Grid de cards — 3 estados da UI + badge cache
frontend/src/services/api.js	✓	Axios centralizado — obterRecomendacoes() e obterEstatisticas()
frontend/src/styles/index.css	✓	Tema preto/dourado Quasar Barber, responsivo mobile
docs/ALGORITMO-E-LOGICA.md	NOVO	Documento técnico completo do algoritmo e arquitetura

16. Decisões Técnicas e Armadilhas

Decisão / Armadilha	Causa	Solução aplicada
Prefixo qb_ nas tabelas	Supabase compartilhado com outro projeto	Prefixo qb_ em todas as 5 tabelas
bigint em vez de UUID	FK UUID → bigint causava erro de tipo	Schema inteiro com bigint GENERATED ALWAYS AS IDENTITY
Supabase JS em vez de Knex	Knex precisa de DATABASE_URL (não disponível no free tier)	@supabase/supabase-js com anon key via PostgREST
RLS bloqueava reads silenciosamente	Novas tabelas têm RLS ativo por padrão — retornava [] sem erro	ALTER TABLE qb_* DISABLE ROW LEVEL SECURITY no SQL Editor
Redis não disponível no Windows	Sem brew, sem apt — Redis não é nativo no Windows	Degradação graceful no cache.js (sistema funciona sem Redis)
upsert falhou no seed-csv.js	qb_clientes não tem UNIQUE em nome_loja — onConflict rejeitado pelo Postgres	Busca existentes primeiro → insert apenas dos novos
Produto duplicado no mesmo pedido (CSV)	Pedido 010 tinha OLEO PARA BARBA 25M - QOD duas vezes	itensAgrupados Map soma quantidade em vez de duplicar linha
Busca exige nome exato do produto	Algoritmo usa nome como chave de string	Frontend exibe placeholder com exemplo; Semana 7 implementa busca parcial

17. Próximos Passos (Semanas 7–12)

Semana	Tema	Principais entregas
7	Gráficos e Visualizações	Chart.js: barras de confiança, scatter lift vs support, distribuição por categoria
8	Testes completos + Deploy	Cobertura >80%, testes de integração API, deploy Vercel + Railway
9	TCC: Intro + Fundamentação	Capítulos 1 e 2, revisão bibliográfica com 5+ artigos, referências ABNT
10	TCC: Metodologia + Implementação	Capítulos 3 e 4, diagramas UML, fluxograma do algoritmo em LaTeX/Word
11	TCC: Resultados + Conclusão	Capítulos 5 e 6, análise das métricas P/R/F1 com dataset completo
12	Apresentação + Defesa	Slides (10-15 slides), ensaio, ajustes finais, submissão à banca

Ações prioritárias para a Semana 7:

- 1. Instalar Chart.js no frontend: `npm install chart.js react-chartjs-2`
- 2. Criar componente `GraficoRegras.jsx` com barras de confiança das top 10 regras
- 3. Adicionar busca parcial no buscador (ILIKE no backend) para facilitar o uso
- 4. Importar histórico completo de vendas (anos anteriores) para melhorar as métricas P/R/F1